

Notions de pointeur en C

1. Définition et notations

Un pointeur est une variable qui peut contenir l'adresse d'une zone mémoire relative ou non à une autre variable, on dit aussi qu'elle pointe sur une variable ou une zone mémoire.

Chaque pointeur déclaré permet de pointer sur un seul type de variables spécifié lors de sa déclaration.

Un pointeur déclaré sur entier ne peut contenir que l'adresse mémoire d'une zone mémoire contenant un entier.

Déclaration	<code>int a; int *pa;</code>
Récupération de l'adresse d'une zone mémoire relative à une variable	<code>pa = &a;</code>
Affichage du contenu de la zone mémoire pointée par pa	<code>printf("%d", *pa);</code>
Modification du contenu de la zone mémoire pointée par pa	<code>*pa = 5;</code>
saisie du contenu d'une zone mémoire pointée par un pointeur	<code>scanf("%d", pa) ; // remarquez l'absence du & dans scanf</code>

En manipulant des variables de type pointeurs, nous devons observer les règles suivantes :

➔ Priorité de * et &

- Les opérateurs * et & ont la même priorité que les autres opérateurs unaires (la négation !, l'incrément ++, la décrémentation --).
- Dans une même expression, les opérateurs unaires *, &, !, ++, -- sont évalués de droite à gauche.

2. Allocation dynamique de la mémoire

Un pointeur contient l'adresse d'une zone mémoire. Cette adresse peut être récupérée par l'opérateur & ou affectée lors de l'exécution du programme grâce aux fonctions d'allocation mémoire.

Les fonctions de gestion de l'espace mémoire se trouvent dans la bibliothèque `stdlib.h`.

Parmi ces fonctions, on distingue :

Fonction	Rôle
malloc prototype : <code>void *malloc(int taille) ;</code>	Renvoie l'adresse de début d'une zone mémoire de longueur taille. La fonction retourne une adresse de type pointeur sur "void" (c'est-à-dire vers un objet de type "non précisé"), qui nécessite l'utilisation de la conversion de type (casting). Exemple : <code>Longueur = 10;</code> <code>s3 = (char *) malloc(longueur * sizeof(char)); :</code> La fonction retourne l'adresse du nouvel espace mémoire alloué ou bien NULL en cas d'échec.
realloc <code>void *realloc(void *p, int NTaille)</code>	change la taille de l'objet pointé par p qui devienne égale à NTaille. La fonction retourne l'adresse du nouvel espace mémoire alloué ou bien NULL en cas d'échec.

calloc <code>void* calloc(int nb_elem, int taille);</code>	La fonction <code>calloc()</code> permet d'allouer de la mémoire et en plus elle initialise à 0 tous les éléments de la zone mémoire. <code>nb_elem</code> = le nombre d'éléments qu'on souhaite pouvoir stocker <code>taille</code> = la taille de chaque éléments que l'on obtient avec <code>sizeof()</code> . En cas d'échec, elle renvoie un pointeur NULL.
free <code>free (void * p);</code>	On utilisera en principe cette fonction autant de fois que la fonction <code>malloc</code> . En effet, elle sert à désallouer ce que l'on avait alloué avec <code>malloc</code> , <code>realloc</code> ou <code>calloc</code>

3. Arithmétique des pointeurs

La valeur d'un pointeur est assimilée à un entier.

On peut appliquer à un pointeur quelques opérations arithmétiques :

- Addition d'un entier à un pointeur, on obtient alors une nouvelle adresse.
- Soustraction d'un entier à pointeur, on obtient alors une nouvelle adresse.
- Différence entre deux pointeurs (de même type), on obtient alors le nombre de valeurs qu'on peut stocker dans l'espace délimité par les deux pointeurs

Remarque

Connaitre une adresse ne permet pas toujours d'accéder à son contenu, il faut que l'espace mémoire désigné par cette adresse ait été réservé, soit par déclaration statique soit par allocation dynamique .

4. Manipulation de tableaux dynamiques

Ce type de tableau est dit tableau dynamique lorsque le nombre de cases réservée n'est connu qu'après le commencement de l'exécution des instructions.

Déclaration	<code>int *T , n ;</code>
Saisie de la taille du tableau	<pre>do { printf ("saisir la taille du tableau : "); scanf (" %d" , &n); } while (n < 1);</pre>
Réservation de l'espace pour les cases du tableau	<pre>T = malloc (n * sizeof (int)); // T contient l'adresse du début du tableau</pre>
Remplissage	<pre>int *p ; // p est déclaré de même type que T for (p = T ; p < (T+n) ; p ++) { printf ("saisir un entier : "); scanf (" %d" , p); // ou & (*p) , & et * s'annulent }</pre>
Affichage	<pre>for (p = T ; p < (T+n-1) ; p ++) { printf (" %d" , *p); }</pre>

Exercice

Ecrire le programme qui permet de remplir un tableau T par n réels saisis au clavier, puis lit un réel X au clavier et détermine et affiche si X appartient ou non à T.

5. Passage des paramètres par variable

La notion de pointeur (adresse) est aussi utilisée en langage C pour permettre de modifier le contenu d'une variable par les instructions d'une fonction autre que celle où cette variable a été déclarée. C'est la notion de passage de paramètre par variable.

Pour réaliser ce traitement, on fournit lors de l'appel l'adresse de la variable qu'on désire permettre de modifier son contenu.

Exemple :

Ecrire le module **rachat** qui permet de modifier à 10 une moyenne qui appartient à l'intervalle [9.80 ; 10[